

RELATÓRIO TRABALHO 1 – MODELAGEM 3D E INTERFACE

DIEGO FERNANDES MOREIRA

ELTON BRASIL LINHARES

MAYCON CHUMBER FERREIRA

DOCENTE: ANTONIO CARLOS DE OLIVEIRA MIRANDA

**RELATÓRIO DE FUNDAMENTOS DE COMPUTAÇÃO GRÁFICA E VISUAL
PROGRAMA DE PÓS-GRADUAÇÃO EM ESTRUTURAS E CONSTRUÇÃO CIVIL
BRASÍLIA, 02 DE JUNHO DE 2026**

FACULDADE DE TECNOLOGIA

UNIVERSIDADE DE BRASÍLIA

1. INTRODUÇÃO

Este trabalho teve como objetivo o desenvolvimento de funcionalidades de modelagem geométrica tridimensional e geração de malhas volumétricas na plataforma 3DModeler.js, desenvolvida como trabalho avaliativo da disciplina Fundamentos da Computação Gráfica e Visual (PECC/UnB). A proposta central foi integrar, em um ambiente interativo baseado na web, a criação de geometrias a partir de perfis bidimensionais e a discretização dessas geometrias em elementos finitos do tipo tetraédrico (T4).

O sistema foi desenvolvido utilizando tecnologias web modernas, com destaque para a biblioteca Three.js para visualização gráfica e o kernel geométrico OpenCascade.js (OCCT/WASM) para operações de modelagem sólida. Além disso, foi implementado um algoritmo próprio de geração de malha, inspirado no comportamento do Netgen, executando inteiramente no navegador sem dependências externas adicionais.

O desenvolvimento abrangeu duas frentes: a tarefa comum a todos os grupos, que consistia na melhoria da interface gráfica, e a tarefa específica do Grupo 1, que exigia a implementação da extrusão de perfis 2D, a geração de malha 3D interna com elementos tetraédricos e a visualização dessa malha no viewport interativo.

2. OBJETIVOS

O desenvolvimento foi orientado pela necessidade de construir um fluxo completo de pré-processamento geométrico e numérico. Os objetivos principais foram:

- Permitir que o usuário desenhe perfis bidimensionais (sketches) no grid;
- Gerar sólidos tridimensionais por extrusão via OCCT/WASM;
- Implementar a geração de malha volumétrica com elementos tetraédricos T4;
- Visualizar a malha no viewport com distinção entre superfície de contorno e estrutura interna;
- Oferecer controle de parâmetros de refinamento comparável a ferramentas como Netgen;
- Melhorar a interface gráfica com ícones, tooltips e identidade visual;

3. SITUAÇÃO DO PROGRAMA BASE

A versão inicial era composta por uma interface de modelagem com ThreeGrid.jsx como componente central. Esse componente coordenava a barra de ferramentas, o canvas 3D, o indicador de modo, os controles de coordenadas, o painel de edição volumétrica e o painel de malha de superfície. A aplicação já possuía uma separação razoável entre interface, visualização e operações geométricas.

No programa base, o usuário já podia trabalhar com ferramentas de desenho, alterar plano de trabalho, controlar grid e snaps, criar curvas, gerar superfícies, utilizar o painel volumétrico e manipular a cena 3D. Entretanto, ainda havia limitações importantes para o objetivo do trabalho: a geração de malha tetraédrica volumétrica não estava consolidada no fluxo FEM, a visualização do sólido podia esconder elementos internos, a seleção de sketch/extrusão não estava suficientemente clara, e não havia um comando simples para excluir sketches ou extrusões já criados.

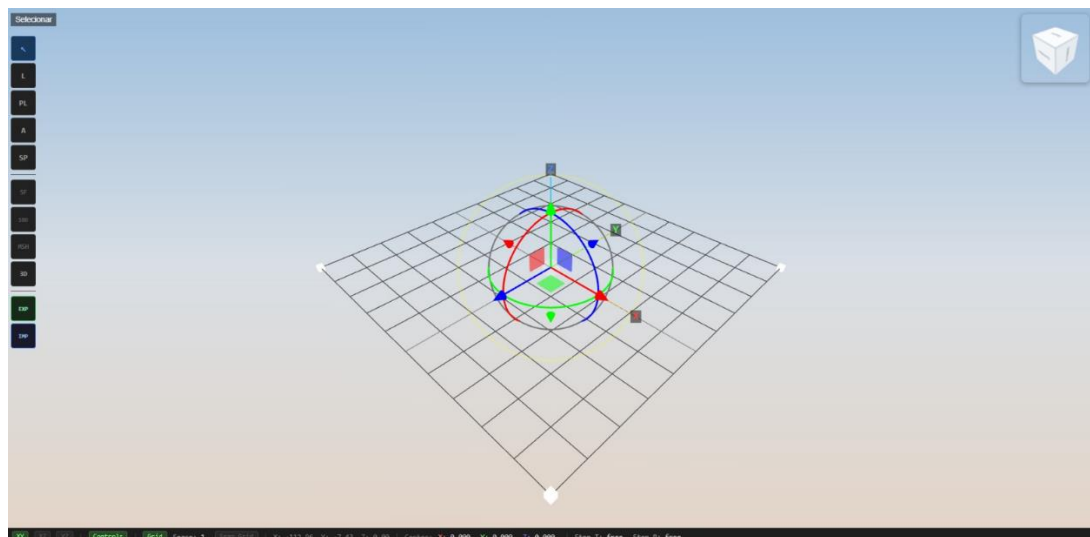


Figura 1 - Interface gráfica do programa base.

4. ARQUITETURA DO SISTEMA

A arquitetura do sistema está organizada em cinco camadas funcionais interligadas por um fluxo unidirecional de dados. Na camada de interface, o componente `Toolbar.jsx` recebe os comandos do usuário e aciona os respectivos callbacks. Em seguida, a camada de visualização 3D, implementada pelo `ThreeCanvas.jsx`, é responsável pelo gerenciamento da cena em `Three.js`, incluindo a seleção de elementos e a aplicação de materiais. A camada de edição geométrica, composta pelos módulos `VolumeEditor.js` e `VolumeEditorAdvanced.js`, realiza as operações de modelagem utilizando a biblioteca OCCT/WASM. Posteriormente, a camada de geração de malha, implementada em `tetMeshUtils.js`, converte as extrusões geométricas em elementos tetraédricos para processamento computacional. Por fim, a camada de histórico, gerenciada pelo componente `VolumeEditorUI.jsx`, mantém o registro das operações executadas, controla o estado da aplicação e coordena as atualizações necessárias entre os diferentes módulos do sistema.

Fluxo principal do programa
Sketch 2D → Superfície ou Extrusão → Sólido 3D (<code>ThreeCanvas</code>)
→ Seleção no histórico (<code>VolumeEditorUI</code>)
→ Geração de malha FEM (<code>SurfaceMeshPanel</code> / <code>tetMeshUtils</code>)
→ Visualização da malha tetraédrica (<code>Three.js</code> <code>LineSegments</code> + <code>Mesh</code>)

5. METODOLOGIA DE MODELAGEM SÓLIDA COM MALHA T4

A metodologia adotada foi baseada na reprodução, em nível conceitual, do fluxo observado em softwares como o FreeCAD. Inicialmente, o usuário desenha um perfil 2D, que é interpretado como um *sketch*. Esse perfil é então processado pelo kernel OpenCascade, que realiza a extrusão ao longo de uma direção definida, gerando um sólido tridimensional.

Após a criação do sólido, inicia-se a etapa de geração da malha. Diferentemente do FreeCAD, que utiliza diretamente o Netgen, foi desenvolvido um algoritmo próprio que busca reproduzir seus principais conceitos. O primeiro passo consiste no pré-processamento do contorno, garantindo que o perfil esteja fechado, sem pontos duplicados e adequadamente representado no plano. Em seguida, o domínio bidimensional é discretizado por meio de uma triangulação de Delaunay, o que permite a geração de uma malha não estruturada, capaz de se adaptar a geometrias arbitrárias.

Para evitar que a malha se restrinja apenas à fronteira, são gerados pontos internos ao domínio, respeitando o contorno definido. Esse procedimento garante o preenchimento completo da região e é fundamental para a formação de uma malha volumétrica consistente. Posteriormente, é aplicada uma etapa de suavização, que ajusta a posição dos nós internos com base na vizinhança, melhorando a qualidade dos elementos e reduzindo distorções.

A malha bidimensional resultante é então extrudada ao longo do mesmo vetor utilizado na geração do sólido. Esse processo cria uma discretização volumétrica em camadas, que é convertida em elementos tetraédricos por meio da subdivisão dos prismas gerados. Além disso, foram implementados procedimentos para identificação das faces externas da malha, permitindo a distinção entre a superfície e o interior do domínio, o que é essencial para a visualização adequada.

5.1. EXTRUSÃO DE PERFIS 2D

A extrusão foi mantida como uma operação central do programa. O fluxo de extrusão depende do sketch ou da superfície selecionada, do vetor de direção e da distância de extrusão. A operação é registrada no histórico para permitir seleção posterior, associação com malha e exclusão.

A principal melhoria não foi apenas gerar o sólido, mas guardar informações suficientes para que esse sólido pudesse ser usado depois na geração da malha. Para isso, o histórico passou a ser essencial. A extrusão registra o identificador do sketch original, a direção de extrusão e os parâmetros necessários para reconstruir o volume lógico.

Esse armazenamento é importante porque a malha tetraédrica precisa saber de onde veio o sólido. Ao selecionar uma extrusão no histórico, o programa consegue localizar o sketch de origem e a direção de extrusão, reconstruindo os dados necessários para gerar a malha volumétrica.

O fluxo de extrusão consiste em produzir um elemento 2D no grid denominado de sketch, no qual é registrado no algoritmo VolumeEditor com seus pontos e configuração de plano, após, o usuário seleciona no menu 3D a ferramenta de criação de volume por extrusão, com isso é definido a distância de extrusão. O sólido é gerado via OCCT/WASM com operação registrada no histórico com todos os dados para reconstrução futura.

5.1.1. Armazenamento de Dados para Malha Posterior

Uma melhoria fundamental foi garantir que o histórico de extrusão contenha todos os dados necessários para gerar a malha tetraédrica posteriormente:

```
...
// src/components/VolumeEditorUI.jsx – recuperação de dados de extrusão para
malha
const generateTetMesh = useCallback((extrudeId) => {
  const ed = edRef.current;
  const scene = getScene();
  if (!ed || !scene) return;

  // Localiza a extrusão no histórico
  const extrudeOp = ed.history.find((op) => op.id === extrudeId);
  if (!extrudeOp || extrudeOp.type !== "extrude") {
    setStatus("Selecione uma extrusão no histórico...");
    return;
  }

  // Recupera o sketch de origem a partir da referência armazenada
  const sketchId = extrudeOp.params?.sketchId ?? extrudeOp.parentId;
  const sketchOp = ed.history.find((op) => op.id === sketchId);
  const sketchPoints = sketchOp?.params?.points;
  const planeConfig = sketchOp?.params?.planeConfig;
  const extrusion = extrudeOp.params?.direction; // THREE.Vector3

  if (!sketchPoints || !planeConfig || !extrusion) {
    setStatus("Dados insuficientes para gerar a malha tetraédrica.");
    return;
  }
  // ... segue para geração da malha
}, [canvasRef, removeTetMeshForId]);
...
```


5.2. GERAÇÃO DE MALHA TETRAÉDRICA VOLUMÉTRICA

5.2.1. Visão Geral do Módulo `tetMeshUtils.js`

O arquivo `tetMeshUtils.js` é o núcleo técnico da implementação. Ele realiza um pipeline completo de tetraedrização volumétrica executando no navegador, usando apenas Three.js e JavaScript puro, sem dependências externas.

A implementação é didática e não substitui geradores profissionais como Netgen ou Gmsh. No entanto, reproduz os conceitos fundamentais de tetraedrização volumétrica com Delaunay 3D, sendo suficientemente robusta para demonstrações acadêmicas.

5.2.2. Presets de Refinamento

Os cinco níveis de refinamento controlam tamanho característico, perturbação aleatória (jitter) e número-alvo de pontos:

```
```javascript
// src/mesh/tetMeshUtils.js – presets de refinamento
const FINENESS_PRESETS = {
 "Muito Grosseiro": { factor: 2.8, jitter: 0.16, smooth: 0, targetPoints:
180 },
 "Grosseiro": { factor: 2.0, jitter: 0.14, smooth: 1, targetPoints:
320 },
 "Moderado": { factor: 1.35, jitter: 0.11, smooth: 2, targetPoints:
520 },
 "Fino": { factor: 0.95, jitter: 0.08, smooth: 2, targetPoints:
760 },
 "Muito Fino": { factor: 0.70, jitter: 0.06, smooth: 3, targetPoints:
980 },
};
// factor → multiplica o tamanho máximo definido pelo usuário
// jitter → perturbação aleatória controlada (evita degeneração de
elementos)
// smooth → iterações de suavização Laplaciana dos pontos internos
// targetPoints → quantidade-alvo de nós na malha gerada
```
```

5.2.3. Triangulação de Delaunay 2D

A primeira etapa do pipeline é a triangulação do contorno 2D do sketch. O algoritmo implementado é o de Bowyer-Watson incremental, que mantém a propriedade de Delaunay:

```

````javascript
// src/mesh/tetMeshUtils.js - teste de circuncírculo (2D)
function circumcircleContains(a, b, c, p) {
 const ax = a.x - p.x, ay = a.y - p.y;
 const bx = b.x - p.x, by = b.y - p.y;
 const cx = c.x - p.x, cy = c.y - p.y;
 const det =
 (ax * ax + ay * ay) * (bx * cy - cx * by)
 - (bx * bx + by * by) * (ax * cy - cx * ay)
 + (cx * cx + cy * cy) * (ax * by - bx * ay);
 const orient = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
 return orient > 0 ? det > 1e-12 : det < -1e-12;
}

// src/mesh/tetMeshUtils.js - Bowyer-Watson 2D
function delaunayTriangulate2D(points) {
 // Cria supertriângulo que engloba todos os pontos
 const work = points.map((p) => p.clone());
 const s0 = work.length, s1 = s0 + 1, s2 = s0 + 2;
 work.push(new THREE.Vector2(midX - 20 * d, midY - d));
 work.push(new THREE.Vector2(midX, midY + 20 * d));
 work.push(new THREE.Vector2(midX + 20 * d, midY - d));
 let tris = [[s0, s1, s2]];

 for (let pi = 0; pi < points.length; pi++) {
 const p = work[pi];

 // Marca triângulos ruins (circuncírculo contém p)
 const bad = [];
 for (let ti = 0; ti < tris.length; ti++) {
 const [a, b, c] = tris[ti];
 if (circumcircleContains(work[a], work[b], work[c], p))
 bad.push(ti);
 }

 // Extrai fronteira do poliedro estrelado e reconstrói com p
 const edgeCount = new Map();
 const addEdge = (a, b) => {
 const key = a < b ? `${a},${b}` : `${b},${a}`;
 edgeCount.set(key, (edgeCount.get(key) ?? 0) + 1);
 };
 for (const ti of bad) {
 const [a, b, c] = tris[ti];
 addEdge(a, b); addEdge(b, c); addEdge(c, a);
 }
 const badSet = new Set(bad);
 tris = tris.filter((_, idx) => !badSet.has(idx));
 for (const [key, count] of edgeCount) {
 if (count !== 1) continue;
 }
 }
}

```

```

 const [a, b] = key.split(",").map(Number);
 tris.push([a, b, pi]);
 }
}
return tris.filter((tri) => tri.every((idx) => idx < points.length));
}
...

```

#### 5.2.4. Tetraedrização 3D de Delaunay

O algoritmo de Bowyer-Watson foi estendido para 3D, operando com tetraedros. O teste de inclusão usa a esfera circunscrita calculada por sistema linear  $3 \times 3$ :

```

```javascript
// src/mesh/tetMeshUtils.js - circumsphere 3D
function circumsphere(a, b, c, d) {
    const rows = [b, c, d].map((p) => [
        2 * (p.x - a.x),
        2 * (p.y - a.y),
        2 * (p.z - a.z),
    ]);
    const rhs = [b, c, d].map((p) =>
        p.x*p.x + p.y*p.y + p.z*p.z - a.x*a.x - a.y*a.y - a.z*a.z
    );
    const den = det3(rows);
    if (Math.abs(den) < 1e-12) return null;
    const center = new THREE.Vector3(
        det3(substituirColuna(rows, rhs, 0)) / den,
        det3(substituirColuna(rows, rhs, 1)) / den,
        det3(substituirColuna(rows, rhs, 2)) / den,
    );
    return { center, radiusSq: center.distanceToSquared(a) };
}

// src/mesh/tetMeshUtils.js - Bowyer-Watson 3D
function delaunayTetrahedralize(points) {
    // Supertetraedro englobante (4 pontos extras)
    work.push(
        new THREE.Vector3(cx,          cy,          cz + 8*d),
        new THREE.Vector3(cx - 8*d,    cy - 6*d,    cz - 4*d),
        new THREE.Vector3(cx + 8*d,    cy - 6*d,    cz - 4*d),
        new THREE.Vector3(cx,          cy + 8*d,    cz - 4*d),
    );
    let tets = [makeTet(work, s0, s1, s2, s3)];

    for (let pi = 0; pi < points.length; pi++) {
        const p = work[pi];
        // Tetraedros ruins: ponto dentro da circun-esfera

```

```

const bad = tets.reduce((acc, t, ti) => {
  const s = t.sphere;
  if (p.distanceToSquared(s.center) <= s.radiusSq * (1 + 1e-9) + 1e-8)
    acc.push(ti);
  return acc;
}, []);

// Extrai faces da fronteira e reconstrói com p
const faceMap = new Map();
for (const ti of bad) {
  const [a, b, c, d0] = tets[ti].ids;
  [[a,b,c],[a,d0,b],[b,d0,c],[c,d0,a]].forEach((f) => {
    const key = faceKey(f);
    const entry = faceMap.get(key);
    if (entry) entry.count++;
    else faceMap.set(key, { count: 1, face: f });
  });
}
tets = tets.filter((_, idx) => !new Set(bad).has(idx));
for (const entry of faceMap.values())
  if (entry.count === 1) {
    const tet = makeTet(work, ...entry.face, pi);
    if (tet) tets.push(tet);
  }
}
return tets.map((t) => t.ids).filter((t) => t.every((i) => i <
points.length));
}
...

```

5.2.5. Função Principal — `generateExtrudedTetraMesh`

A função exportada é a interface pública do módulo. Recebe os parâmetros de extrusão, monta a nuvem de pontos, executa a tetraedrização e retorna geometrias Three.js prontas para adicionar à cena:

```

```javascript
// src/mesh/tetMeshUtils.js – função principal exportada
export function generateExtrudedTetraMesh({
 sketchPoints, // pontos do contorno 2D (THREE.Vector3[])
 planeNormal, // normal do plano do sketch
 extrusion, // vetor de extrusão (THREE.Vector3)
 maximumSize = 5,
 fineness = "Moderado",
 secondOrder = false,
 // ...outros parâmetros
} = {}) {
 const contour3D = sanitizeContour(sketchPoints);

```

```

const extLen = ext.length();
const { u, v } = buildPlaneBasis(normal); // base ortonormal do plano

// Projeta contorno 3D no plano local 2D
const contour2D = contour3D.map((p) => {
 const rel = p.clone().sub(origin);
 return new THREE.Vector2(rel.dot(u), rel.dot(v));
});

// Calcula tamanho efetivo respeitando orçamento de pontos
const preset = FINENESS_PRESETS[fineness] ??
FINENESS_PRESETS.Moderado;
const budgetSize = Math.cbrt(volume / targetPoints) * 0.92;
const effectiveSize = Math.max(requestedSize, budgetSize);

// Gera nuvem de pontos 3D e aplica Delaunay volumétrico
let cloud = buildLocalPointCloud({ contour2D, extLen, h: hUsed, preset });

// Proteção de desempenho: aumenta h se pontos demais
let guard = 0;
while (cloud.points.length > hardPointLimit && guard < 8) {
 hUsed *= 1.18;
 cloud = buildLocalPointCloud({ contour2D, extLen, h: hUsed, preset });
 guard++;
}

// Filtra tetraedros fora do volume
const linearTets = delaunayTetrahedralize(cloud.points).filter((tet) => {
 const centroid = avg(tet.map((i) => cloud.points[i]));
 return centroid.z >= 0
 && centroid.z <= extLen
 && pointInPolygon(centroid.xy, planar.boundary);
});

// Retorna geometrias Three.js prontas para a cena
return {
 vertices, tets, shellGeometry, wireGeometry,
 nNodes: vertices.length,
 nTets: tets.length,
 elemOrder: secondOrder ? 2 : 1,
 method: "Tetraedrização volumétrica 3D",
 qualityWarning, effectiveSize,
 // ...demais metadados
};
}
...

```

### 5.2.6. Renderização da Malha — `createTetraMeshObject`

O objeto Three.js da malha é um grupo com dois filhos: uma mesh translúcida para a superfície de contorno e um LineSegments para as arestas internas (wireframe), permitindo visualizar a densidade real da discretização:

```
```javascript
// src/mesh/tetMeshUtils.js - criação do objeto Three.js
export function createTetraMeshObject(meshData, options = {}) {
  const { shellColor = 0x60a5fa, edgeColor = 0x1d4ed8,
    shellOpacity = 0.18 } = options;
  const group = new THREE.Group();
  group.name = "tetra-volume-mesh";

  // Casca translúcida: faces de fronteira da malha
  const shell = new THREE.Mesh(
    meshData.shellGeometry,
    new THREE.MeshStandardMaterial({
      color: shellColor,
      transparent: true,
      opacity: shellOpacity,
      side: THREE.DoubleSide,
      depthWrite: false,
      roughness: 0.9,
    })
  );

  // Wireframe: todas as arestas internas - permite ver o refinamento real
  const wire = new THREE.LineSegments(
    meshData.wireGeometry,
    new THREE.LineBasicMaterial({
      color: edgeColor, transparent: true, opacity: 0.95,
    })
  );

  group.add(shell);
  group.add(wire);
  group.userData.meshStats = {
    nNodes: meshData.nNodes,
    nTets: meshData.nTets,
    method: meshData.method,
  };
  return group;
}
```
```

## 6. MELHORIA DA INTERFACE GRÁFICA

### 6.1. ÍCONES SVG NAS FERRAMENTAS

Todas as ferramentas da barra de comandos passaram a usar ícones SVG desenhados programaticamente, substituindo os rótulos de texto simples da versão base. Os ícones foram definidos diretamente no arquivo `Toolbar.jsx` como elementos JSX, sem dependência de bibliotecas externas de ícones.

Exemplo real extraído do arquivo — ícone da operação Extrudar (`Toolbar.jsx`, linhas 13-22):

|                                                                                       |
|---------------------------------------------------------------------------------------|
| <b>src/components/Toolbar.jsx — definição de ícone SVG (Extrudar)</b>                 |
| <code>const CREATE_OPS = [</code>                                                     |
| <code>{</code>                                                                        |
| <code>  id: "extrude",</code>                                                         |
| <code>  label: "Extrudar",</code>                                                     |
| <code>  icon: (</code>                                                                |
| <code>    &lt;svg width="16" height="16" viewBox="0 0 16 16"</code>                   |
| <code>      fill="none" stroke="currentColor"</code>                                  |
| <code>      strokeWidth="1.3" strokeLinecap="round" strokeLinejoin="round"&gt;</code> |
| <code>        &lt;rect x="3" y="8" width="10" height="6" rx="1" /&gt;</code>          |
| <code>        &lt;line x1="8" y1="8" x2="8" y2="2" /&gt;</code>                       |
| <code>        &lt;polyline points="5.5,4.5 8,2 10.5,4.5" /&gt;</code>                 |
| <code>      &lt;/svg&gt;</code>                                                       |
| <code>  ),</code>                                                                     |
| <code>  },</code>                                                                     |
| <code>  // ... (revolve, loft, siga-me, entre-faces — mesmo padrão)</code>            |
| <code>];</code>                                                                       |

O mesmo padrão foi aplicado às operações booleanas (União, Subtração, Interseção) com ícones de diagrama de Venn, e às ferramentas de desenho 2D (seleção, aparar, snap, grid).

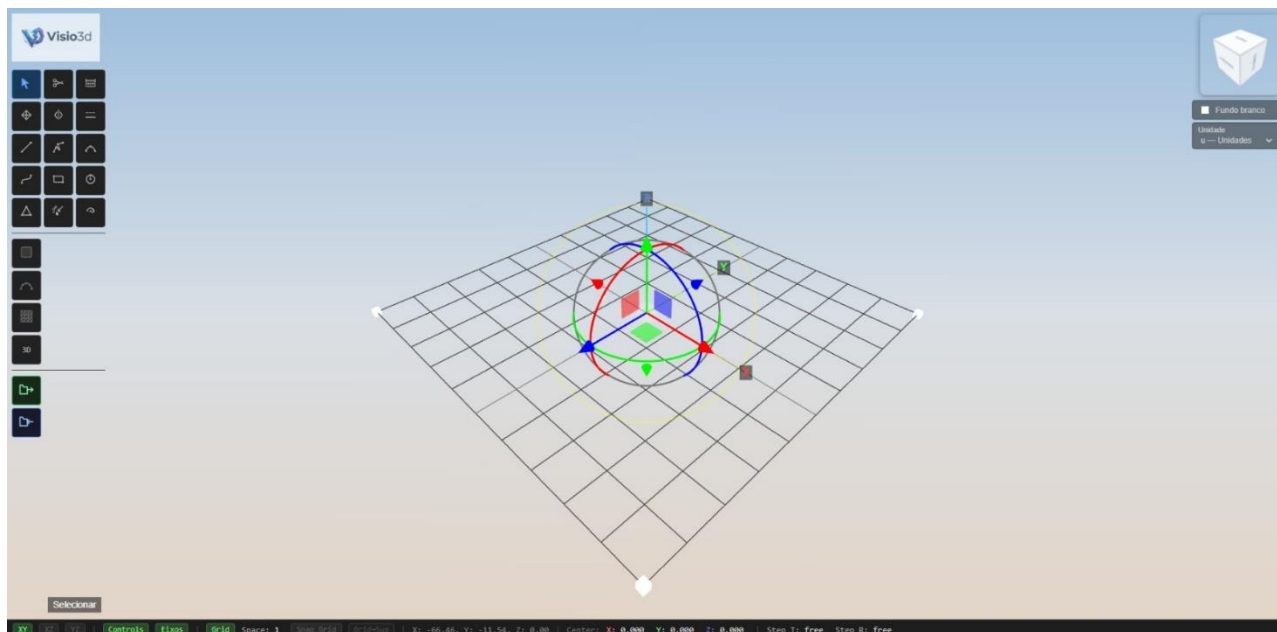


Figura 2 - Nova interface gráfica.

## 6.2. TOOLTIPS E ORIENTAÇÃO CONTEXTUAL

Todos os botões da barra de ferramentas receberam atributos title com descrições funcionais. O campo de distância da ferramenta Linha tem dica contextual específica, e os painéis de volume recebem mensagens de status que orientam o próximo passo do fluxo de trabalho.

| src/components/Toolbar.jsx — botões com tooltip (title) |
|---------------------------------------------------------|
| {TOOL_ROWS.map((row, rowIdx) => (                       |
| <div key={rowIdx} style={{ display: 'flex', gap: 4 }}>  |
| {row.map(({ id, label, title }) => {                    |
| const active = activeTool === id;                       |
| return (                                                |
| <button                                                 |
| key={id}                                                |
| title={title} // tooltip nativo do navegador            |
| onClick={() => onToolChange(id)}                        |
| style={{                                                |
| background: active ? '#1a3a5c' : '#222',                |
| color: active ? '#6af' : '#aaa',                        |



|                                                                   |
|-------------------------------------------------------------------|
| <code>border: `1px solid \${active ? '#4a7abf' : '#444'}`,</code> |
| <code>...</code>                                                  |
| <code>}}</code>                                                   |
| <code>&gt;</code>                                                 |
| <code>{label} // label agora é um SVG, não texto</code>           |
| <code>&lt;/button&gt;</code>                                      |
| <code>);</code>                                                   |
| <code>}}}</code>                                                  |
| <code>&lt;/div&gt;</code>                                         |
| <code>));</code>                                                  |

### 6.3. LOGOTIPO E IDENTIDADE VISUAL

O arquivo logo.png foi adicionado à raiz do projeto frontend e referenciado no topo da Toolbar para compor a identidade visual da plataforma. A barra passou a ter cor escura (#111-#1a2a3a) com destaque azul nos elementos ativos, criando uma aparência consistente de ferramenta CAD.



Figura 3 - Logo.

## 7. DESAFIOS ENFRENTADOS E SOLUÇÕES ADOTADAS

| # | Desafio                                                                                     | Solução adotada                                                                                                                            |
|---|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Fusão de versões divergentes do projeto: arquivos modificados em paralelo geravam conflitos | Estratégia de fusão incremental: interface mais completa como base, reinserção da lógica de malha nessa base, comparação arquivo a arquivo |
| 2 | Sólido extrudado opaco ocultava sketch de origem e malha interna                            | Material alterado para transparente com opacity: 0.18; seleção indicada por cor/emissive, nunca por opacidade total                        |
| 3 | Alteração do nível de refinamento não produzia variação visual perceptível                  | Causa: falta do wireframe interno. Solução: LineSegments com todas as arestas T4; debounce de 250ms para regeneração automática            |
| 4 | Gerar malha em um segundo objeto apagava a malha do primeiro                                | Substituição de estado global por Map<extrudeId, THREE.Group>, operando sempre na entrada correspondente                                   |
| 5 | Excluir sketch deixava extrusão referenciando geometria inexistente                         | Algoritmo de propagação iterativa em deleteOperation resolve dependências em cascata antes de remover                                      |
| 6 | Malha tetraédrica no painel errado (Objeto 3D vs FEM)                                       | Transferida para SurfaceMeshPanel.jsx — painel FEM — separando criação geométrica de discretização                                         |

## **8. USO DE INTELIGÊNCIA ARTIFICIAL NO DESENVOLVIMENTO**

## 9.

Conforme previsto e incentivado no enunciado da atividade, ferramentas de Inteligência Artificial foram empregadas como suporte técnico ao longo de todas as etapas do desenvolvimento, atuando como instrumento auxiliar à tomada de decisão da equipe, sem substituir a análise crítica e o julgamento dos integrantes. A IA contribuiu para a análise das diferentes versões do projeto, auxiliando na identificação dos arquivos que deveriam ser preservados, mesclados ou substituídos durante o processo de integração. Também apoiou a definição da estratégia de fusão adotada, baseada na preservação da versão com interface mais completa e na posterior reintegração da lógica de geração de malha. Durante a fase de testes, a ferramenta foi utilizada no diagnóstico de problemas, auxiliando na formulação de hipóteses para comportamentos observados no navegador, como ausência de variação visual da malha, desaparecimento de elementos ou problemas de renderização dos sólidos. Além disso, colaborou na geração e revisão de trechos de código, incluindo a elaboração de versões modificadas dos arquivos do sistema e do módulo `tetMeshUtils.js`, posteriormente avaliadas, ajustadas e validadas pela equipe.

A IA também foi utilizada como apoio na organização e redação da documentação técnica do projeto. Em todas as etapas, os testes funcionais e visuais foram conduzidos pelos integrantes da equipe, responsáveis por verificar os resultados obtidos, indicar correções necessárias e validar o comportamento final da aplicação.

## **10.LIMITAÇÕES CONHECIDAS**

Apesar dos resultados obtidos, algumas limitações e oportunidades de melhoria foram identificadas durante o desenvolvimento. A geração da malha tetraédrica foi implementada com finalidade didática utilizando JavaScript diretamente no navegador, não apresentando o mesmo nível de robustez, desempenho e confiabilidade de ferramentas consolidadas de geração de malhas, como o Netgen ou o Gmsh, especialmente para o tratamento de geometrias complexas. Além disso, componentes centrais do sistema, como `ThreeCanvas.jsx` e `VolumeEditorUI.jsx`, acumularam um número elevado de responsabilidades e linhas de código, o que pode dificultar a manutenção, a evolução e os testes da aplicação, tornando recomendável sua futura refatoração em módulos menores e mais especializados. Outra limitação observada é a ausência de mecanismos para exportação das malhas geradas em formatos amplamente utilizados em análises por elementos finitos, como VTK, Abaqus INP e CGNS, restringindo a interoperabilidade com softwares externos. Durante o processo de integração das versões também foram produzidos arquivos temporários com extensão `.bak`, os quais devem ser removidos antes da disponibilização da versão final do projeto. Por fim, verificou-se que geometrias contendo concavidades complexas podem demandar algoritmos adicionais de filtragem e validação para garantir a remoção adequada de tetraedros externos e a obtenção de uma malha mais consistente e representativa do volume modelado.

## 11. CONCLUSÃO

O Grupo 1 implementou todas as funcionalidades previstas no Trabalho 1: melhoria da interface gráfica com ícones SVG, tooltips e logotipo (tarefa comum), e o fluxo completo de modelagem sólida com extrusão de perfis 2D, geração de malha tetraédrica volumétrica T4 com algoritmo de Delaunay 3D e visualização no viewport (tarefa específica).

A versão atual do 3DModeler.js permite criar sketches 2D, gerar extrusões registradas no histórico, visualizar sólidos translúcidos com destaque de seleção, excluir operações com propagação de dependências em cascata, manter múltiplas malhas independentes por objeto e controlar o refinamento via presets com regeneração automática.

O desenvolvimento foi conduzido de forma incremental: cada teste visual revelou um problema específico, e cada problema levou a uma solução documentada. O resultado é um programa com identidade própria, interface organizada e capaz de demonstrar o ciclo completo entre modelagem geométrica 3D e discretização tetraédrica para análise de elementos finitos, executando inteiramente no navegador sem dependências externas adicionais.

## REFERÊNCIAS

- Three.js Documentation — <https://threejs.org/docs/>
- OpenCascade.js — <https://github.com/donalfons/opencascade.js>
- React Documentation — <https://react.dev>
- Vite Build Tool — <https://vitejs.dev>
- Bowyer, A. (1981). Computing Dirichlet Tessellations. The Computer Journal, 24(2), 162-166.
- Watson, D. F. (1981). Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes. The Computer Journal, 24(2), 167-172.
- Enunciado do Trabalho — Plataforma 3DModeler.js, PECC/UnB, 2026